CPSC 538-G Research Project Final Report

Symbolic and Statistical Evaluation of the CARTEL Consensus Algorithm Using UPPAAL

Sean Donaghy | 935 919 23

Introduction

A significant challenge in designing real-time systems is verifying the correctness of the specifications of those systems to establish invariant properties and timing guarantees required of real-time applications. Multiple real-time system verification methods exist with benefits and tradeoffs for each. This work focuses on evaluating the properties and required assumptions for Consensus Real-Time Efficient Logging (CARTEL) [1], a recently proposed enhancement to the Raft Consensus Algorithm [2] which seeks to improve the timing predictability of Raft election and commit operations. The CARTEL authors validate their protocol by testing an implementation of it on a small cluster of real hardware. This work aims to validate the CARTEL protocol by instead using the formal specification and model checking tool for real-time systems, UPPAAL 5.0 [3]. The intent is for this work to serve as a case study for how UPPAAL can be used to verify real-time distributed system designs with a higher degree of controllability and fewer physical resources than the hardware stress testing carried out in the CARTEL study.

The two modifications that CARTEL makes to the standard Raft algorithm are: CARTEL voting (to ensure consistent leader election timing) and CARTEL node buffers (to bound log commit timing). This work focuses solely on the election protocol with the key contribution being an investigation and demonstration of how the CARTEL election protocol can be modelled and evaluated against standard Raft, using networks of timed automata with UPPAAL. This report, along with the accompanying models and model checker queries, provides hardware independent verification and evaluation of the CARTEL election protocol and the findings reported by the authors.

Background

Analytical Verification of Real Time Systems

To verify the correctness of a real-time system design, designers often use analytical methods such as response time analysis or utilization bounds to rigorously prove that their system satisfies

some set of mathematical properties. However, these methods can often lead to complex and elaborate analyses which may still miss edge cases and lead to incorrect timing guarantees [4].

Stress Testing

To minimize the probability of undetected edge cases, designers may also stress test their system on real hardware. While stress tests can provide some level of assurance that a system is unlikely to violate timing properties, they cannot provide timing guarantees nor be used to rigorously prove any other correctness properties at the semantic level. Additionally, given that stress tests must be run on real hardware, they do not provide a way for the logic of the system or protocol to be validated in isolation from any implementation details or environmental factors. This also leads to limited controllability over factors such as transmission delay or interference from other processes during stress testing.

Formal Specification and Verification

Formal specification is a method which allows designers to logically specify the key properties and behaviors of a system while abstracting away any implementation details and ignoring external influences. The goal with formal specification is to rigorously define what a system *should* do while remaining as agnostic as possible to *how* it should do it. This approach allows a designer to cleanly separate the semantics of a system from its implementation to evaluate and communicate the logic of that system in isolation from its concrete implementation.

Symbolic Model Checking

With a formal specification of a system in hand, a designer can use a model checker to assert that a variety of correctness properties hold for their system. Symbolic model checkers work by exhaustively searching the entire state space of a system and asserting that all specified correctness properties are satisfied. In the context of hard realtime systems, these properties may be firm timing guarantees or other invariants or safety properties that must be true either eventually or in all states. Model checking also provides a high degree of controllability over factors that can be difficult to constrain in a real implementation, such as jitter in network latency or task interference in a non-realtime scheduling regime. Having better controllability can also allow a designer to force events which may be difficult to reproduce in concrete implementations, such as race conditions or rare event sequences.

Statistical Model Checking

While an exhaustive state space search can provide strong guarantees, it is not always feasible due to the combinatorial expansion of state space size for even moderately complex models. This can be especially problematic for models of realtime systems, as they often contain many clock variables which can quickly make an exhaustive search intractable. Statistical model checkers,

on the other hand, avoid exhaustive state space exploration by instead running several simulations of the modeled system to verify correctness properties with a certain level of confidence [5]. This approach affords much more complex, expressive, and realistic models with the compromise of weaker guarantees than from symbolic model checkers.

UPPAAL

To model both the Raft and CARTEL election protocols, this work uses the specification and model checking toolbox, UPPAAL. Standard UPPAAL can be used to symbolically model a realtime system with a network of finite state machines extended with clock variables, custom functions, and synchronization channels. This work uses UPPAAL version 5.0 which is also bundled with UPPAAL SMC (Statistical Model Checker), which provides support for defining stochastic models and statistical model checker queries such as probability and expected value estimation. The argument for using UPPAAL in this work is that it lets us conduct an implementation and hardware agnostic evaluation of the CARTEL election protocol, with enhanced controllability over event ordering and network latency compared to the real-world testing setup used by CARTEL authors. UPPAAL also serves as a much lower cost system validation framework compared to the cluster of several high-performance compute modules used in the CARTEL study. Finally, we can use UPPAAL's built-in statistical analysis tools and model checker query engine to rapidly iterate on analyses and symbolic/statistical correctness checks.

Raft

The Raft algorithm enables nodes in a distributed system to reach consensus on a replicated log, which is used to keep their state machines mutually consistent. A Raft system achieves consensus by electing a leader who then proposes log entries, which are replicated to followers and considered committed once a majority of followers agree. As a consequence of this design, if the leader node fails, the system cannot make progress until a follower node times out waiting to hear from the leader, transitions to a candidate state, and starts a new election term to become the new leader. While the Raft algorithm has been proven to be correct, the original specification offers little in the way of timing guarantees, specifically lacking an upper bound on leader election latency. For example, in the event of a split vote, a new election term must be started, and system progress is further delayed. The Raft authors note that if follower election timeouts are selected randomly and uniformly (the authors suggest the range 150-300 ms) then split vote events should be very unlikely. The CARTEL authors do confirm this to be true, and in the average case Raft is able to elect a new leader very quickly. However, CARTEL authors note two key drawbacks of this approach. Firstly, they claim that even the small possibility of an unbounded election latency due to repeated split votes can be unacceptable for certain realtime distributed applications. Secondly, they show that split vote events can also occur even when follower nodes initiate elections at different times, due to the variance in transmission delay of vote requests from candidate nodes to other nodes in the system.

Consensus Real Time Adaptive Efficient Logging (CARTEL)

To address the problem of unbounded worst case election latency in standard Raft, the CARTEL authors propose the CARTEL voting strategy. CARTEL voting prevents split votes during an election by ensuring each node waits until it has received vote requests from all candidates in the current election, thus preventing unbounded election latency. This is achieved by ensuring that upon receiving the first vote request of a new election term, each node waits for a period called a *pendingTimeout* before voting for the candidate with the shortest election timeout. This method relies on the assumption that the maximum network transmission delay is known, and still that no two candidate nodes have the same election timeout. The CARTEL authors analytically show that if the *pendingTimeout* is in the range:

$$2 \cdot \delta \leq pendingTimeout < ET^{\min} - 2 \cdot \delta \qquad (1)$$

Where δ is the maximum network delay and ET^{min} is the minimum election timeout, and no two candidates have the same election timeout, then a leader is guaranteed to be elected in a single term with no split vote. The key point to emphasize for the CARTEL voting scheme is that it provides an upper bound, which could not be given for standard Raft, of the worst-case election latency. The authors argue this can be more important, for some realtime systems, than having a shorter election latency in the average case. In fact, when the authors benchmark CARTEL voting against standard Raft, they found that Raft reelects a new leader faster than CARTEL on average, but that CARTEL reelects a new leader several seconds faster than Raft in the worst case, where Raft experiences some split votes while CARTEL does not.

Methodology

Three UPPAAL models were created for this study in the following order:

1. **raft:** An idealized model of the standard raft election state machine that assumes zero network delay and instant synchronization across nodes. To minimize the state space, this model uses a small election timeout range of 5-to-10-time units.
2. **raft_time_delay:** A more realistic model with asynchronous message passing, randomly sampled message transmission delays, and wider timeout ranges (150-to-300-time units).
3. **cartel_time_delay:** A copy of raft_time_delay but with the CARTEL voting strategy instead of standard Raft voting.

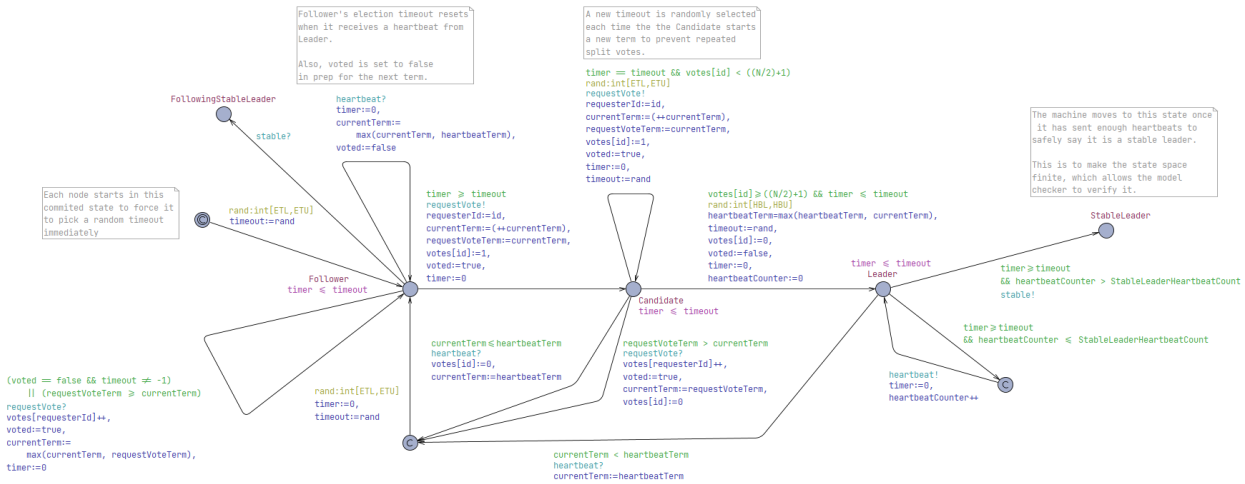


Figure 1. raft: The timed automaton representing the election protocol for standard Raft.

To create the **raft** model shown in figure 1, this work used an existing example model [6] and made substantial upgrades to improve its testability and adherence to the Raft specification, including improving logic for candidates and leaders stepping down, fixing a bug in election winner determination, and adding terminal states to assert the system reaches a stable state. With the **raft** model, it was possible to validate that we had a correct baseline implementation of the election protocol by querying UPPAAL's symbolic model checker for multiple safety, liveness, and reachability properties. Given the simplistic assumptions of **raft,** it was feasible to run exhaustive checks to confirm that: the system would not reach deadlock (liveness), the system would eventually find a leader (reachability), and for every state where there are two leaders there will eventually be only one (safety).

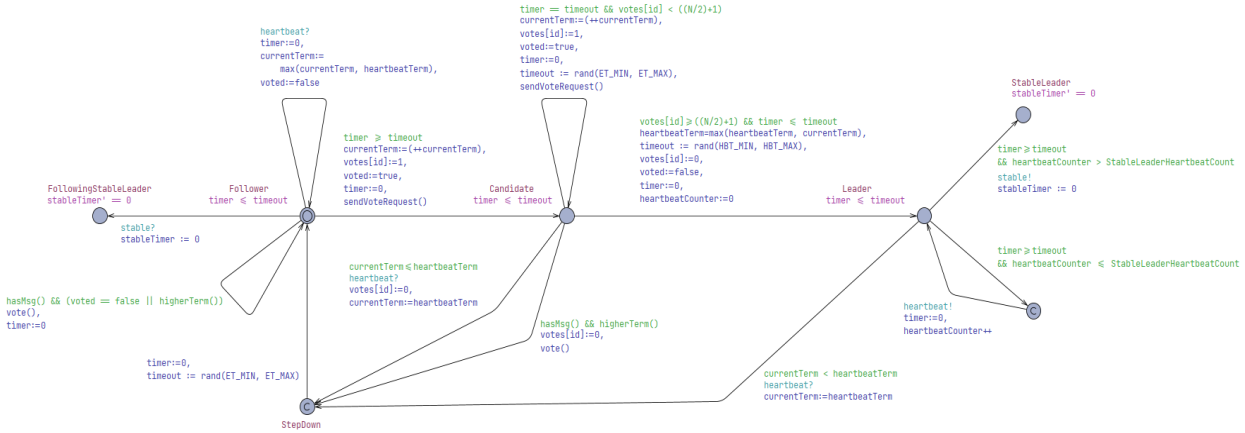


Figure 2. raft_time_delay-Node: The timed automaton representing the standard Raft election protocol, extended with asynchronous message passing with randomized delays.

While **raft** was a useful starting point for validating modelling techniques, the assumption of zero-delay message passing made it unfit for realistically representing a real distributed system

or testing the timing characteristics of the CARTEL election protocol. The model **raft_time_delay** (Figure 2) was created to add more realistic delays between when candidates send vote request messages and when those messages arrive at each node. To implement asynchronous message passing in a system of N nodes, **raft_time_delay** represents each vote request message as a tuple:

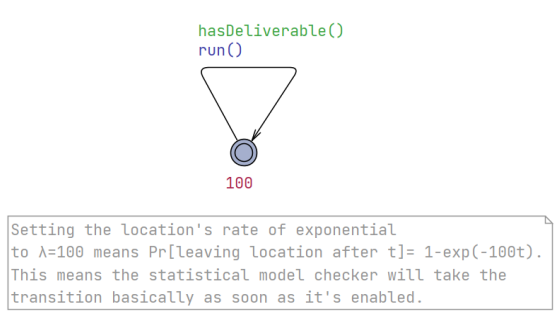
$$\{src, dest, netDelay, netTimer, delivered\}$$

where src is the candidate node sending the vote request, $dest$ is the node to receive the request, $netDelay$ is a random uniformly selected value in the range 20-40 ms, $netTimer$ is a clock variable which acts as a stopwatch, recording how much time has passed since the message was sent, and $delivered$ is a flag that is set to **0** when a message is sent and **1** once $netTimer \geq netDelay$. The network delay is sampled from the range 20-40 ms because a uniform sampling of this range gives an expected one-way latency of 30ms which aligns with the mean round trip latency of ~60ms measured by the CARTEL authors in their stress test setup. For the simulations in this study, we also choose a *pendingTimeout* of 40 ms which satisfies Eq. 1.

```
/**
    sendVoteRequest: sends voteRequestRPCs to all other nodes by setting msg[id][i] = 1
                     for all i in {0,.., id-1, id+1, ...,N-1}
*/
void sendVoteRequest() {
    for (i : int[0,N-1]) {
      netTimer[id][i] = 0;
      if (i ≠ id) {
        candidateTerms[id] = currentTerm;
        candidateTimeouts[id] = timeout;
        msg[id][i] = 1;
        delivered[id][i] = 0;
        netDelay[id][i] = rand(NET_DELAY_MIN, NET_DELAY_MAX);
       }
    }
}
```

Figure 3. The *sendVoteRequest()* function broadcasts a vote request message to all other nodes, selecting an independent random amount of delay for each message.

For a system with N nodes, vote request messages are tracked in an $N \times N$ array called *msg* where setting *msg*[i][j] equal to 1 is semantically equivalent to node i sending a vote request message to node j, as shown in figure 3.




```
void run() {
    for (i : int[0,N-1]) {
      for (j : int[0,N-1]) {
        if (msg[i][j] == 1 && delivered[i][j] == 0 && netTimer[i][j] ≥ netDelay[i][j]) {
          delivered[i][j] = 1;
        }
      }
    }
}

bool hasDeliverable() {
  for (i : int[0,N-1]) {
      for (j : int[0,N-1]) {
          if (msg[i][j] == 1 && delivered[i][j] == 0 && netTimer[i][j] ≥ netDelay[i][j])
              return true;
      }
  }
  return false;
}
```

Figure 4. The Network automaton (left) and logic (right)

Nodes continuously check the $delivered$ flags to detect when vote request messages that have been sent to them have arrived. The work of monitoring all the $netTimer$s and marking inflight messages as delivered is managed by an auxiliary automaton called the Network (figure 4). The Network continuously checks if there are any in flight messages that were sent at least $netDelay$ time units ago by evaluating the edge guard function *hasDeliverable()*. If there are any such messages, the automaton marks those messages as delivered in the *run()* function. For an N node system the $N \times N$ array *delivered* keeps track of whether in-flight messages have arrived. In the *run()* function, setting *delivered*[i][j] equal to 1 is equivalent to the vote request from node i arriving at node j. It is not possible to construct an automaton that is constantly firing an edge while also allowing time to pass, therefore the Network must go to sleep (by staying in its initial state) and wake up (fire the *run()* edge) periodically to allow system time to pass during simulations. This means the Network may introduce added latency as a modelling artifact in both **raft_time_delay** and **cartel_time_delay**, but it should not significantly affect the comparisons between Raft and CARTEL simulations.

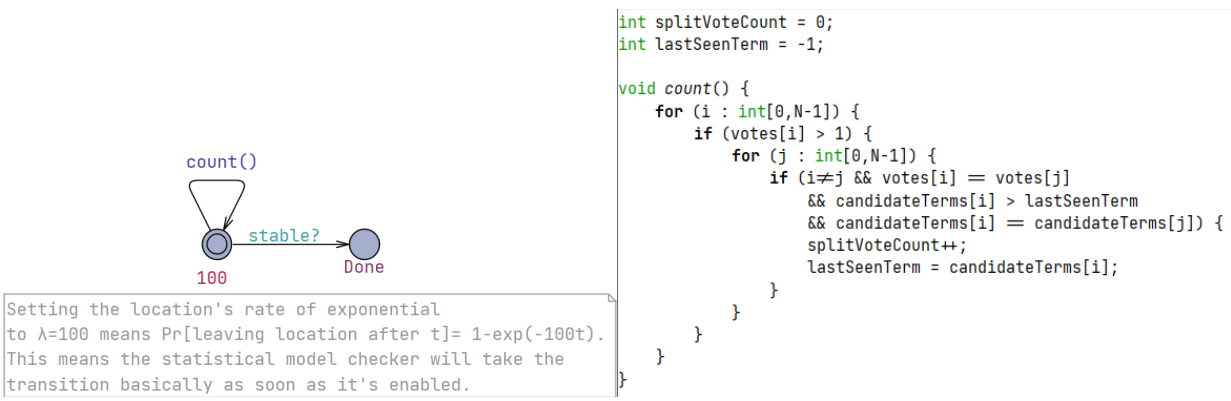


Figure 5. The SplitVoteCounter automaton (left) and *count()* function (right). The terminal state Done is reached when the system reaches a stable state. Adding this state makes it possible for the SplitVoteCounter to reach a final state which is necessary to query the splitVoteCount value from a simulation.

In addition to the Network, another auxiliary automaton, called the SplitVoteCounter (figure 5), continuously audits the system to detect when split votes occur, trying not to double count split votes in the same election term. The SplitVoteCounter is used later in analysis to track the number of split votes in simulations and to verify no split votes occur in simulations of **cartel_time_delay**.

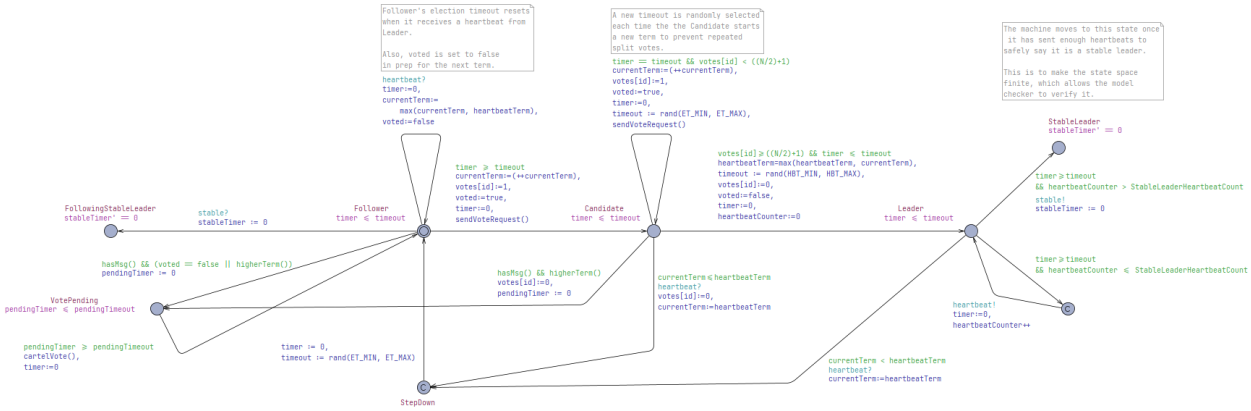


Figure 6. cartel_time_delay-Node: The timed automaton representing the CARTEL election protocol.

The third and final model created for this study is **cartel_time_delay** (figure 6), which is the same as **raft_time_delay** except with CARTEL voting instead of standard Raft voting. The key difference in this model is that when the automaton receives a vote request, rather than immediately voting, it moves to an intermediate state named VotePending where it waits for a *pendingTimeout* before taking the transition back to the Follower state and voting for the candidate with the smallest election timeout, which follows the CARTEL voting specification. With the realistic timing characteristics of the **raft_time_delay** and **cartel_time_delay** comes the inevitability that the state spaces of these models are infeasible to exhaustively search with UPPAAL's symbolic model checker. Therefore, the target with these models is not to verify the same properties as we did for **raft**, but rather to run a series of simulations on each, in order to estimate probabilities of events of interest, estimate expected values for leader election time, and demonstrate the frequency of split votes.

The CARTEL authors' test setup involved running trials of leader election scenarios for both standard Raft and CARTEL using two different election timeout ranges of [1s, 2s) and [1.5s, 2.5s) for a total of four tests, running on a cluster of seven nodes. To verify the findings of these tests, this work reproduces the same tests using the simulation tool UPPAAL SMC (Statistical Model Checker) [5]. The scenario for every simulation is to launch the system with all nodes initially in the follower state, as if a leader failure had just occurred, and then allowing the system to run until it reaches a stable state.

Stable State

This study defines system stability as once a single node has been able to send fifty heartbeat messages as a Leader without stepping down due to receiving any heartbeats or vote requests from other nodes in a higher election term. Once this occurs, the system is deemed stable and all nodes move to their respective terminal states to conclude the simulation. The value of fifty was chosen heuristically as a large enough value to be quite confident that if there are multiple

leaders, fifty cycles should be enough for them to detect each other and for all but one Leader to step down.

The notion of a stable state is another contribution of this work, which extends the analysis of the CARTEL paper. In their tests, the CARTEL authors only measure the latency of some node becoming a Leader first. However, having a node become Leader is not a concrete indicator of system stability because it is entirely possible for two nodes to be Leaders simultaneously, for a brief period of time, before one steps down. This is because the Raft specification permits at most one Leader in a single election term but allows for multiple simultaneous Leaders in different terms under some fault scenarios such as network partitioning or delayed messaging. To assert that both the **raft_time_delay** and **cartel_time_delay** models do reach a stable state the following statistical query was run on both models:

```
Pr[≤15000](<>
        node0.StableLeader
        + node1.StableLeader
        + node2.StableLeader
        + node3.StableLeader
        + node4.StableLeader
        + node5.StableLeader
        + node6.StableLeader == 1)
```

This query asks the model checker to estimate the probability that exactly one node eventually becomes the StableLeader (and thus a stable state is reached) in less than 15 seconds. For both models this probability was estimated to be ≥ 0.95 with 95% confidence. To measure the time, it takes for each simulation to reach a stable state, a timer called *stableTimer* is configured to start at the beginning of each simulation and stop as soon as the nodes transition to their stable states. The SMC is then queried with:

```
E[≤15000; 10000] (max: stableTimer)
```

which runs 10,000 simulations, for up to 15 seconds each, and estimates the maximum time to reach stability with 95% confidence. We also define the term stability latency to be the time it takes for a system to reach a stable state starting from $t = 0$ when all nodes are initially Followers.

Results

These plots of simulation data were generated using UPPAAL's plot composer tool.

Split Votes

Query: simulate[≤10000; 100] {counter.splitVoteCount}

<<100 simulations, 10 seconds each, report number of split votes in each simulation>>

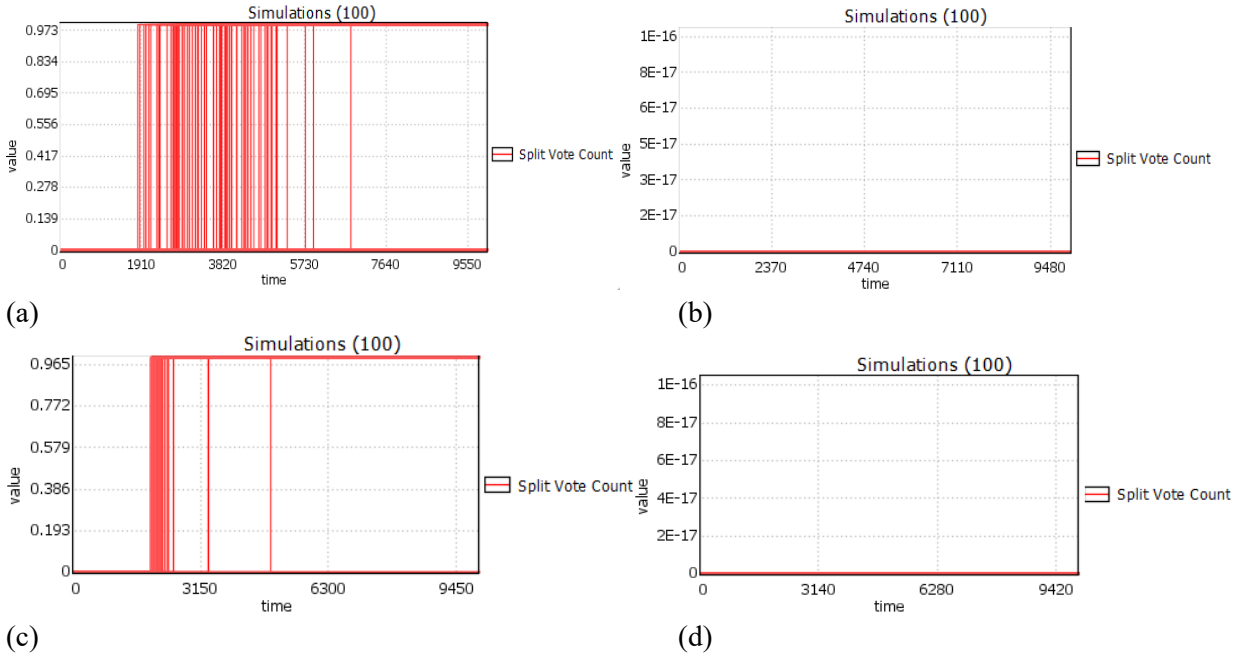
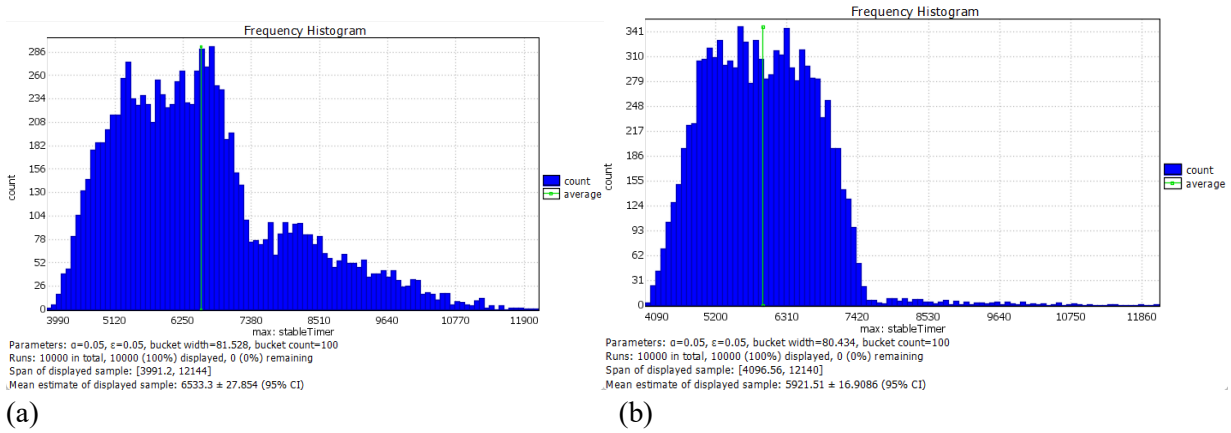


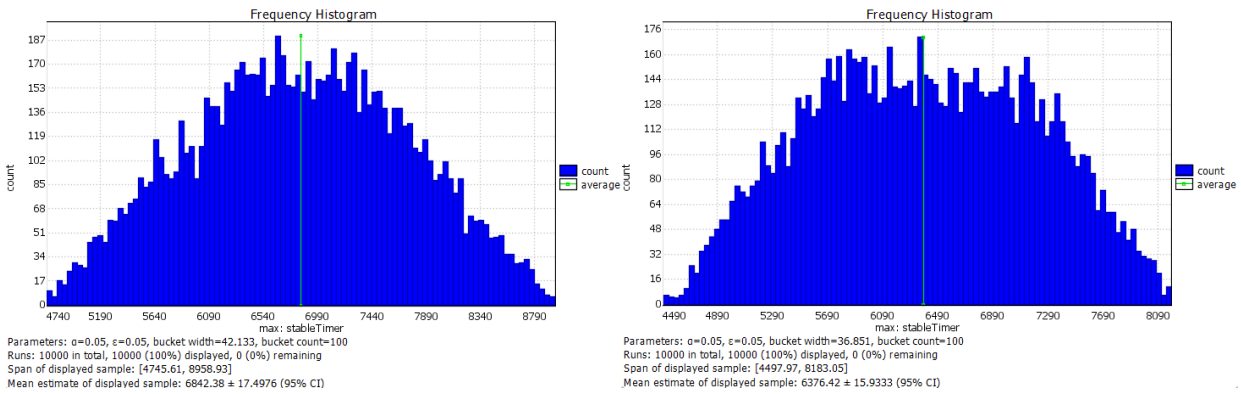
Figure 7. Number of recorded split vote events in one hundred simulations of the standard Raft election algorithm (left) and CARTEL election algorithm (right) with [1s, 2s) (top) and [1.5s, 2.5s) (bottom) election timeouts. Each red line is a count of the number of split votes for an individual simulation run. Units along the x axis are in milliseconds.

Time To Reach Stable State (Stability Latency)

Query: E[≤15000; 10000] (max: stableTimer)

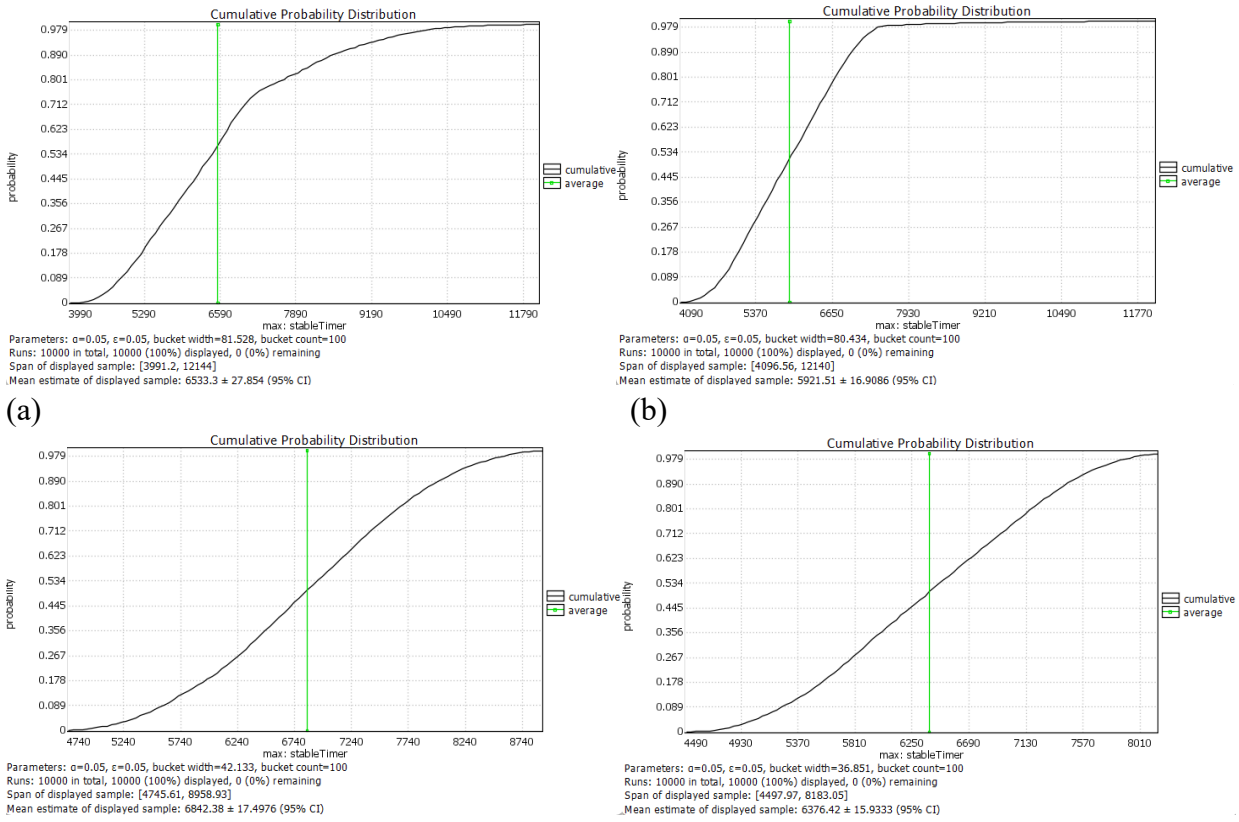
<<10,000 simulations, 15 seconds each, estimate the maximum time to reach stability>>





(c) (d)

Figure 8. Distribution of time to reach stability in 10,000 simulations of the standard Raft election algorithm (left) and CARTEL election algorithm (right) with [1s, 2s) (top) and [1.5s, 2.5s) (bottom) election timeouts. Each bar represents how many of the simulations reached stability within the time interval of the corresponding bucket. Units along the x axis are in milliseconds.



(a) (b)

(c) (d)

Figure 9. Cumulative probability distribution of time to reach stability in 10,000 simulations of the standard Raft election algorithm (left) and CARTEL election algorithm (right) with [1s, 2s) (top) and [1.5s, 2.5s) (bottom) election timeouts. Units along the x axis are in milliseconds.

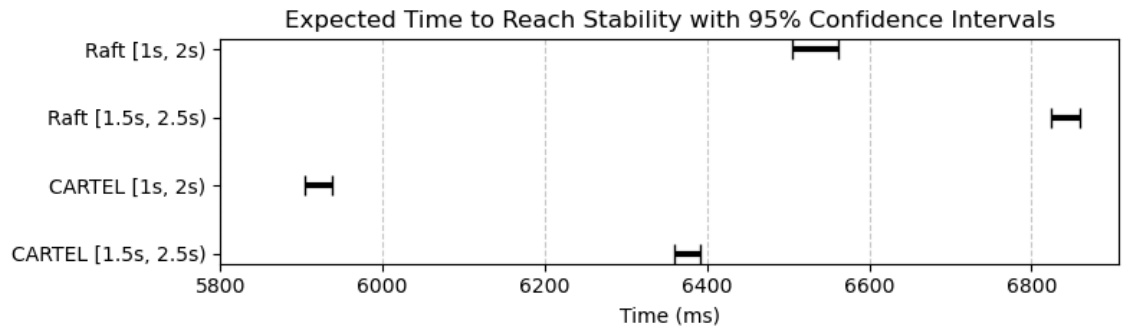


Figure 10. 95% confidence intervals for expected time to reach a stable state for Raft and CARTEL with [1s, 2s) and [1.5s, 2.5s) election timeout ranges.

Discussion

As expected, many of the standard Raft simulations experienced split vote occurrences, whereas no split votes occurred in any simulations for the CARTEL election algorithm. This of course, agrees with the safety property outlined by the CARTEL authors, that split votes will not occur provided the pending timeout satisfies Eq. 1. Interestingly, no more than a single split vote occurs in all the simulations of Raft, indicating the rareness of the multiple repeated split votes event.

The distribution of times at which split votes occur also noticeably changes between the [1s, 2s) (fig 7.a) and [1.5s, 2.5s) (fig 7.c) election timeout tests of Raft. In the [1.5s, 2.5s) case, the split vote events are clumped densely around 2-3 seconds while the split vote occurrences are slightly more spread out for [1s, 2s). While I do not have a definite explanation for this phenomenon, it does show that the range of the election timeouts has interesting effects on the dynamics of the system. Perhaps the election timeouts being even larger than the network delay reduces the number of collisions.

The histograms for time to reach system stability demonstrate some intriguing insights as well. Firstly, the distributions of time to reach stability for the [1s, 2s) election timeout cases for Raft and CARTEL are a picture-perfect representation of what CARTEL voting aims to achieve. We observe that the distribution in figure 8.a is right skewed with a heavy tail. We expected this because standard Raft is quick to find a new leader in the average case but can have a high tail latency in the worst case due to split vote events causing elections to be restarted. On the other hand, figure 8.b shows the improved timing predictability of CARTEL voting. In the histogram (fig 8.b) we observe a clear point in time around 7400 ms, before which most of the simulations reached a stable state. This clearly defined upper bound on the latency of reaching stability is the main benefit that CARTEL voting provides over standard Raft, and it has been successfully demonstrated with these simulations. Figure 9.b shows the cumulative probability distribution for the stability latency of the same set of simulations and demonstrates the same clear point

around t = 7400 ms. The way to interpret Figure 9.b is that the estimated probability of CARTEL reaching stability in less than 7400 ms is very close to one.

It should also be mentioned that there are some outlier simulations in figure 8.b, with stability latencies beyond 7400 ms. The cause of these outliers is due to how the Network automaton functions. To allow time to pass between instances of the Network "waking up" and firing its *run()* edge, the Network's single state needed to be a normal state (not committed or urgent) and thus the frequency at which the Network fires its *run()* edge is inherently random. This means the Network can add some of its own latency to message passing which could potentially accumulate and lead to some of the outlier latencies we observe in the histogram.

Next, we observe that in the histograms and CDFs for the simulations of Raft and CARTEL run with [1.5s, 2.5s) election timeouts, we see a vastly different pattern than for the [1s, 2s) case. Oddly, both algorithms have remarkably similar distributions, suggesting that the longer election timeouts affect the system dynamics in such a way that Raft and CARTEL voting perform more similarly. This is a surprising finding and not something detected or discussed by CARTEL authors. While the exact mechanism for this behavior is not clear, it highlights the critical role that the choice of election timeout range plays in designing a consensus algorithm with predictable timing characteristics. One possible explanation is that when election timeouts are significantly larger than the network delays the system is more stable and the likelihood of split votes is reduced, thus hiding the benefits of CARTEL voting. Another perspective on the same idea, is that, in the [1.5s, 2.5s) case, the pendingTimeout in CARTEL represents a smaller proportion of the total election timeout. As a result, the additional delay introduced by pendingTimeout becomes less significant relative to the overall election duration, making CARTEL's behavior more similar to Raft. This interpretation is supported by the observation that both the mean and median stability latencies (Figures 8.c and 8.d) are closer between Raft and CARTEL in the [1.5s, 2.5s) case than in the [1s, 2s) case, indicating a convergence in their performance under longer timeout ranges.

Given that (at least for the [1s, 2s) election timeout simulations) we have a clear empirical upper bound on stability latency for CARTEL, we can determine how close this value is to the theoretical expectation.

In the CARTEL system **cartel_time_delay**, the maximum amount of time it should take to go from Leader failure to electing a new Leader is given by:

$$\begin{aligned}\Delta t_{\text{Leader Fail}\rightarrow\text{New Leader}} &= \text{ElectionTimeout}^{\max} + \text{NetDelay}^{\max} + \text{pendingTimeout} \\ &= \text{ElectionTimeout}^{\max} + \text{NetDelay}^{\max} + 2 \cdot \text{NetDelay}^{\max} \\ &= 2000 + 3 \cdot (40) \\ &= 2120\end{aligned}$$

The maximum time it should take to go from a Leader to a StableLeader is given by:

$$\begin{aligned}\Delta t_{\text{Leader}\to\text{StableLeader}} &= \text{HeartbeatTimeout}^{\max} \cdot (\text{StableHeartbeatCount} + 1)\\ &= 100 \cdot (51)\\ &= 5100\end{aligned}$$

Therefore, the maximum time we expect CARTEL to take to reach stability is:

$$\begin{aligned}\Delta t_{\text{Leader Fail}\to\text{StableLeader}} &= \Delta t_{\text{Leader Fail}\to\text{New Leader}} + \Delta t_{\text{Leader}\to\text{StableLeader}}\\ &= 2120 + 5100\\ &= 7220\end{aligned}$$

which is only about 2% off from the empirically observed bound of ~7400 ms.

A final key takeaway is that CARTEL reaches stability faster than Raft, **in expectation**, in both the [1s, 2s) and [1.5s, 2.5s) election timeout cases. This is shown in figure 10, which displays the 95% confidence intervals for expected stability latency of both Raft and CARTEL with [1s, 2s) and [1.5s, 2.5s) election timeout ranges. This is most certainly the result of Raft having a higher tail stability latency and skewing its mean latency values higher. Given nodes can vote right away with Raft, we still expect Raft to typically recover faster from leader failures, due to it having a lower median/typical stability latency than CARTEL.

Conclusion

This study set out to analyze and evaluate the timing characteristics of the newly proposed CARTEL voting algorithm using formal verification methods and statistical model checking. We were able to produce realistic models of both the standard Raft and CARTEL election protocols using networks of timed automata in UPPAAL and were able to run a series of parameterized statistical simulations on those models to reproduce some expected results as well as new insights into the dynamics of the CARTEL voting algorithm. Through the simulation results, we were able to demonstrate that CARTEL provides a firmer upper bound on leader election latency than standard Raft by way of eliminating the occurrence of split votes during leader election. We also observed that whether or not the two algorithms exhibit distinct behavior is dependent on the choice for election timeout range, with Raft and CARTEL simulations in the [1.5s, 2.5s) election timeout range showing more similarities in stability latency distribution than for the [1s, 2s) election timeout case. Additionally, we were able to show that the theoretical upper bound on stability latency for CARTEL in our models (7220 ms) was within approximately 2% of the empirically measured value (7440 ms).

Given that these models and simulations were built and run on a single machine, and using a free UPPAAL academic license, this work also stands as a successful case study on how CARTEL and other realtime distributed system designs can be tested and validated early on in the development cycle and with little upfront cost compared to building a concrete implementation and expensive multi machine stress testing setup.

Future work may include extending the **cartel_time_delay** model to replicate the CARTEL node buffering feature [1] for predictable log commit timing, improving the realism of the models by adding randomized network delay to vote casting and heartbeat messages, and sampling network latencies from a lognormal distribution rather than uniformly.

References

[1] S. Lee, W. Lee, S. Cho, S. Lee, and J. Lee, "CARTEL: Consensus adapting real-time and efficient logging," in *Proceedings of the 2025 IEEE Real-Time Systems Symposium (RTSS)*, Dec. 2025, pp. 461–473. IEEE.

[2] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conference (USENIX ATC)*, 2014, pp. 305–319.

[3] I. Grobelna, K. Gajewski, and A. Karatkevich, "A systematic review on the applications of UPPAAL," *Sensors*, vol. 25, no. 11, Art. no. 3484, 2025, doi: 10.3390/s25113484.

[4] R. I. Davis, A. Burns, R. J. Bril, *et al*., "Controller Area Network (CAN) schedulability analysis: Refuted, revisited and revised," *Real-Time Systems*, vol. 35, no. 3, pp. 239–272, 2007, doi: 10.1007/s11241-007-9012-7.

[5] A. David, K. G. Larsen, A. Legay, M. Mikučionis, and D. B. Poulsen, "Uppaal SMC tutorial," *International Journal on Software Tools for Technology Transfer*, vol. 17, no. 4, pp. 397–415, 2015.

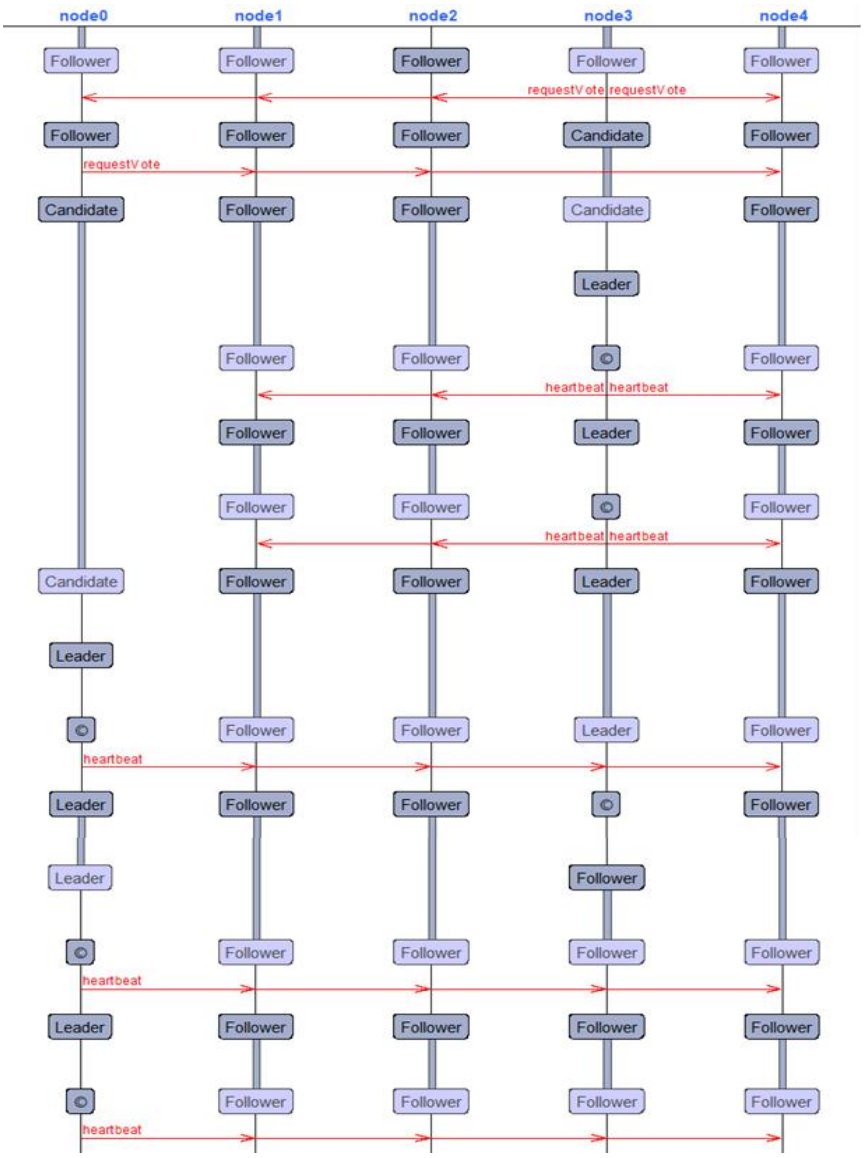
[6] psjung, "Raft-Uppaal," GitHub repository. [Online]. Available: Raft-Uppaal repository. Accessed: Apr. 12, 2026.

Appendix

Link to UPPAAL models used in this study

https://github.com/SPDonaghy/raft_uppaal/tree/main

Tail Probability Analysis on Time to Reach Stability

Probability of Reaching Stability in ≤ 7220ms and ≤ 8000ms	
Raft [1s, 2s)	Pr[Reach Stability in ≤ 7220 ms] ∈ [0.950, 1.000] (99.9% CI)
Raft [1.5s, 2.5s)	Pr[Reach Stability in ≤ 8000 ms] ∈ [0.709, 0.805] (99.9% CI)
CARTEL [1s, 2s)	Pr[Reach Stability in ≤ 7220 ms] ∈ [0.950, 1.000] (99.9% CI)
CARTEL [1.5s, 2.5s)	Pr[Reach Stability in ≤ 8000 ms] ∈ [0.926, 0.996] (99.9% CI)

Example trace of a system having two leaders and then one